

Multimodal visual-page fallback

When a query targets a specific document and page, the retrieval system bypasses standard database thresholds to execute an offline text-extraction. The raw text layer is read directly from the local file and injected as high-value virtual hits prior to re-ranking. During synthesis, any hits flagged by this fallback trigger an on-the-fly rendering pass. The system generates a high-resolution screenshot of the requested page and feeds it directly into the model alongside the text, creating a true multimodal context.

Programmatic-Only Activation Criteria

Importantly, **no command-line flags** exist on the standard CLI commands (`rag query` or `rag chat`) to pass page filters or document-level restrictions. To invoke the multimodal visual-page fallback, you must call the underlying search or synthesis functions **programmatically** via our Python API. This allows evaluation harnesses and downstream applications to isolate queries with extreme spatial precision.

Work example:

The following Python code snippet demonstrates how to import `answer_question` and format the API call to pass document and page filters, mirroring our evaluation harness implementation:

```
from src.rag.inventory import load_inventory
from src.rag.query import answer_question
from src.rag.retrieve_defaults import DEFAULT_LAYER_WEIGHT

# 1. Load the active project inventory
inv = load_inventory()

# 2. Configure target document and page-level constraints
target_doc_id = "3048_CAD_159825_ZUSAMMENBAU"
target_pages = [1] # List of 1-based page numbers

# 3. Format and execute the query programmatically
prediction = answer_question(
    question="Locate the hydraulic pump assembly block and specify its part number.",
    inv=inv,
    alpha=DEFAULT_LAYER_WEIGHT,
    expand=False,
    from_images=True, # Enable visual-dominant retrieval preset
    machine="3048",
```

```
        doc_id=target_doc_id,  
        page_filter=target_pages  
    )  
  
    print(prediction)
```